

Designing Topology-Aware Collective Communication Algorithms for Large Scale InfiniBand Clusters: Case Studies with Scatter and Gather

Krishna Kandalla, Hari Subramoni, Abhinav Vishnu and Dhabaleswar K. (DK) Panda

IPDPS 2010

Lecture 24
April 13, 2026



Credit: IBM

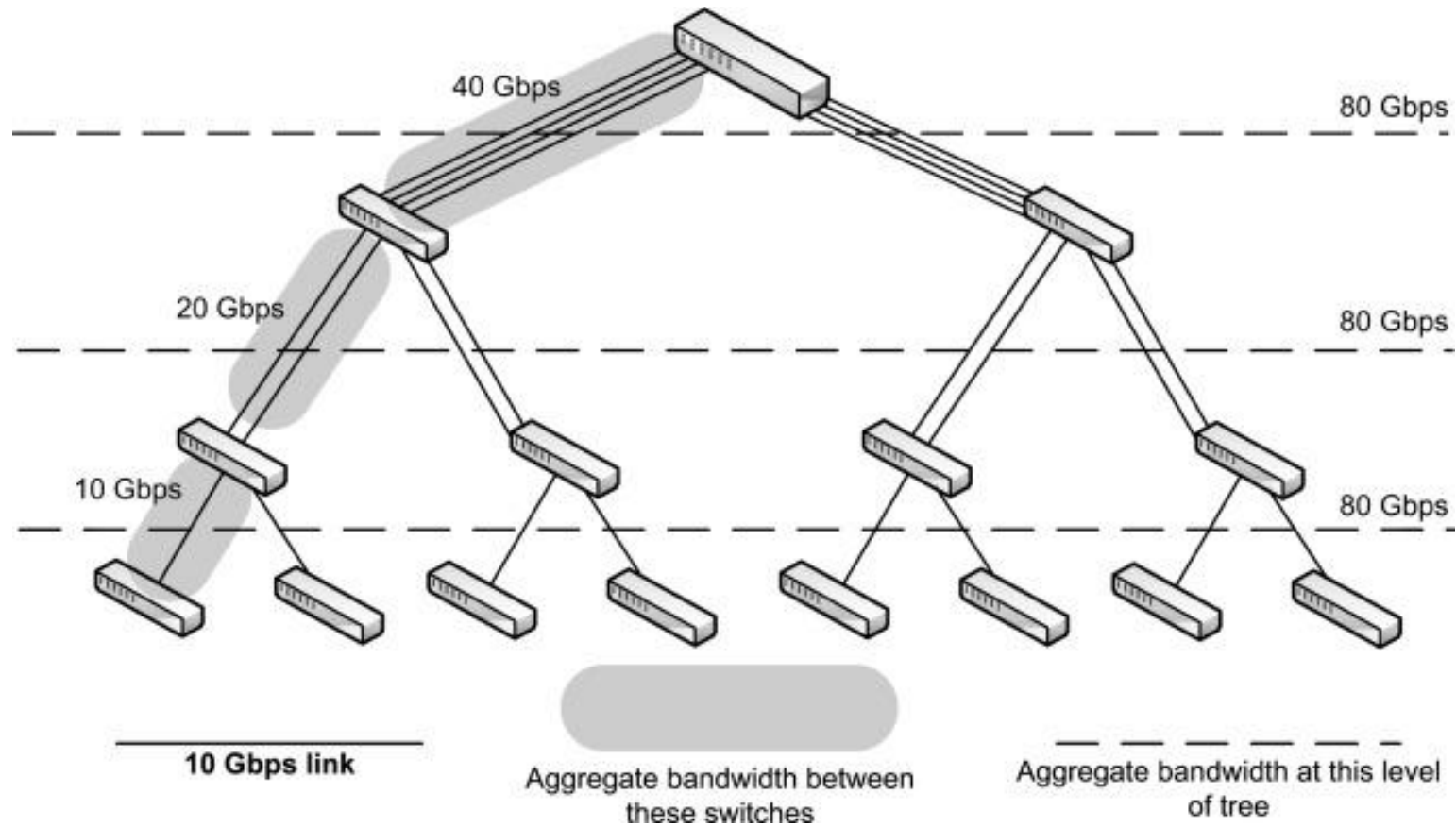
Abstract

Modern high performance computing systems are being increasingly deployed in a hierarchical fashion with multi-core computing platforms forming the base of the hierarchy. These systems are usually comprised of multiple racks, with each rack consisting of a finite number of chassis, and each chassis having multiple compute nodes or blades, based on multi-core architectures. The networks are also hierarchical with multiple levels of switches. Message exchange operations between processes that belong to different racks involve multiple hops across different switches and this directly affects the performance of collective operations. In this

Research Questions

- How do we efficiently **discover the topology** of a large scale InfiniBand cluster?
- What are the challenges involved in designing **efficient** collective algorithms that are **aware of the network topology**?
- Can we derive **communication cost models** for collective operations on large-scale systems with several levels of hierarchies?
- What is the **effect of the background traffic** on the performance of collective operations? Can we leverage the topology information to design algorithms that are **resilient** to network contention?
- This paper specifically analyzes MPI_Scatter and MPI_Gather

Network Topology



Fat-tree topology

[www.sciencedirect.com]

A typical topology of
large-scale systems
(TACC Ranger system)

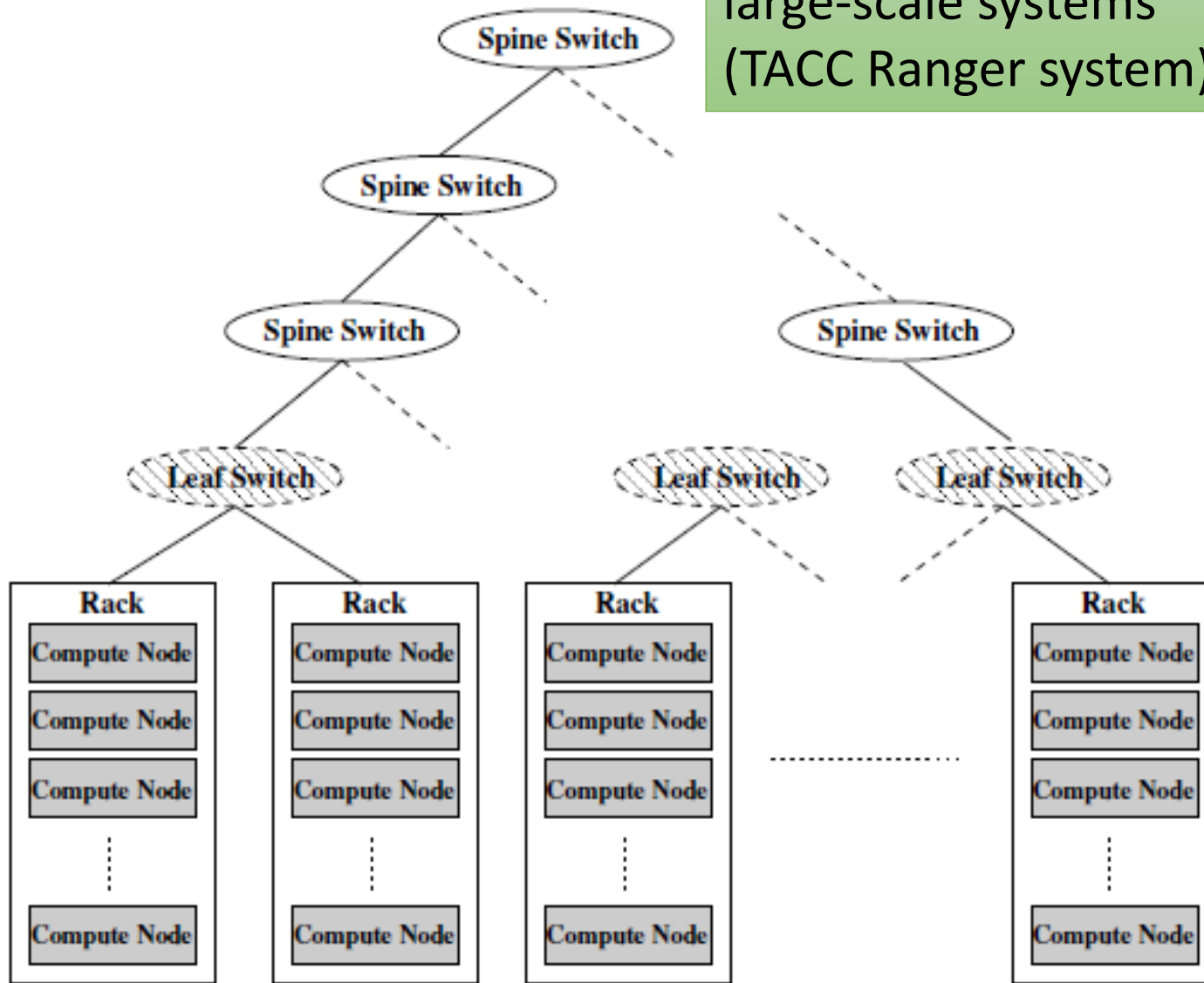


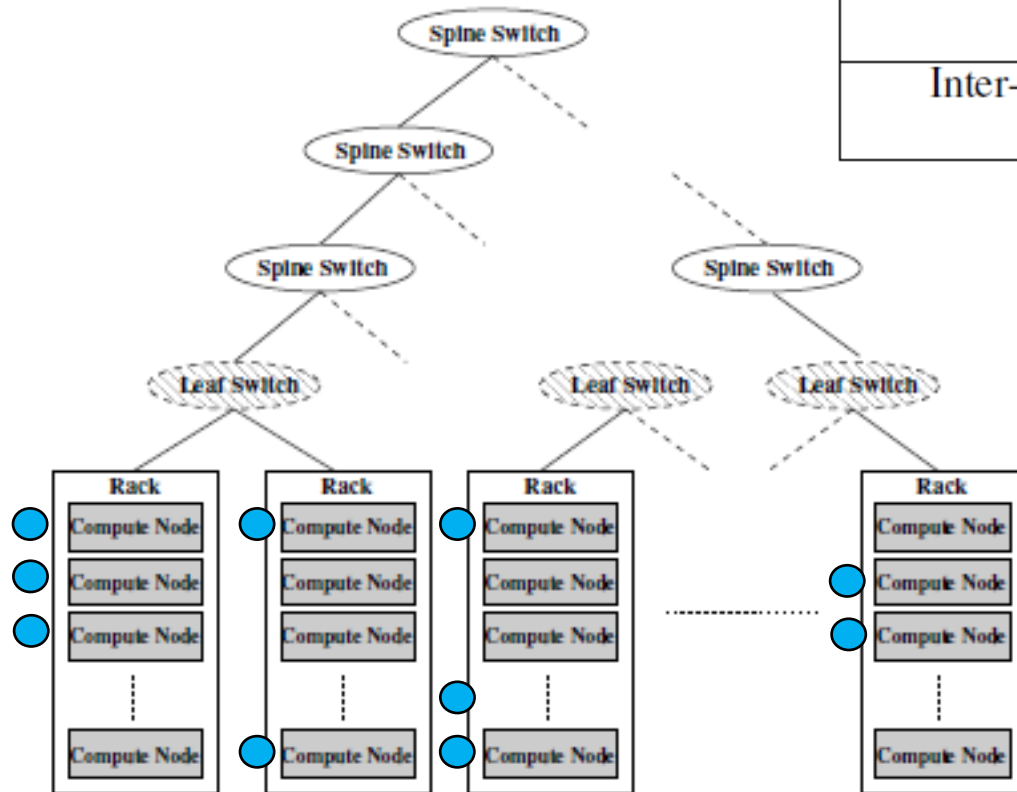
Figure 1. A Typical Topology

Challenges

- Thousands of compute nodes
- Core density is on the rise
- Nodes coupled through hierarchy of switches
- Operate at near-peak load
- Limits effective network bandwidth available per job
 - Processes of a job are allocated across racks

Effect of Topology on Latency

Process Location		Number of Hops	MPI Latency (<i>us</i>)
Intra-Rack	Intra-Chassis	0 Hops in Leaf Switch	
	Inter-Chassis	1 Hop in Leaf Switch	
Inter-Rack		3 Hops Across Spine Switch	
		5 Hops Across Spine Switch	



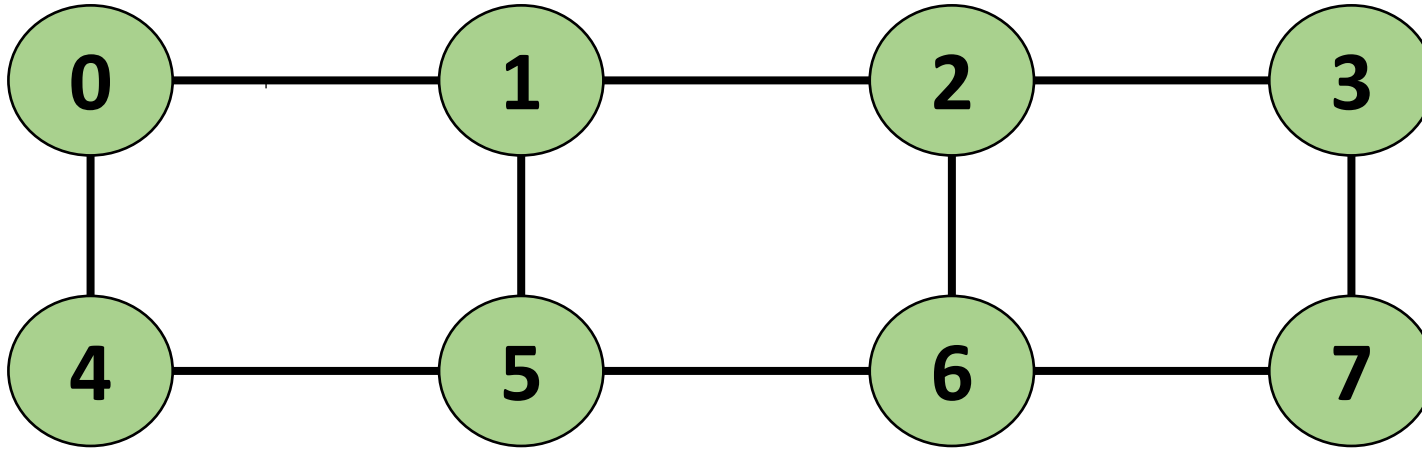
- Allocated nodes are usually scattered in the system
- Different job request sizes and durations
- Contiguous node allocation may increase queue waiting times

A typical topology of large-scale systems
(TACC Ranger system)

Contribution: Topology-aware Collectives

- Topology-aware MPI_Gather and MPI_Scatter
- Modified communication cost model
- 54% improvement on micro-benchmarks

MPI_Bcast Binomial Communication Cost Recap



0 -> 4

$L + n/B$

0 -> 2, 4 -> 6

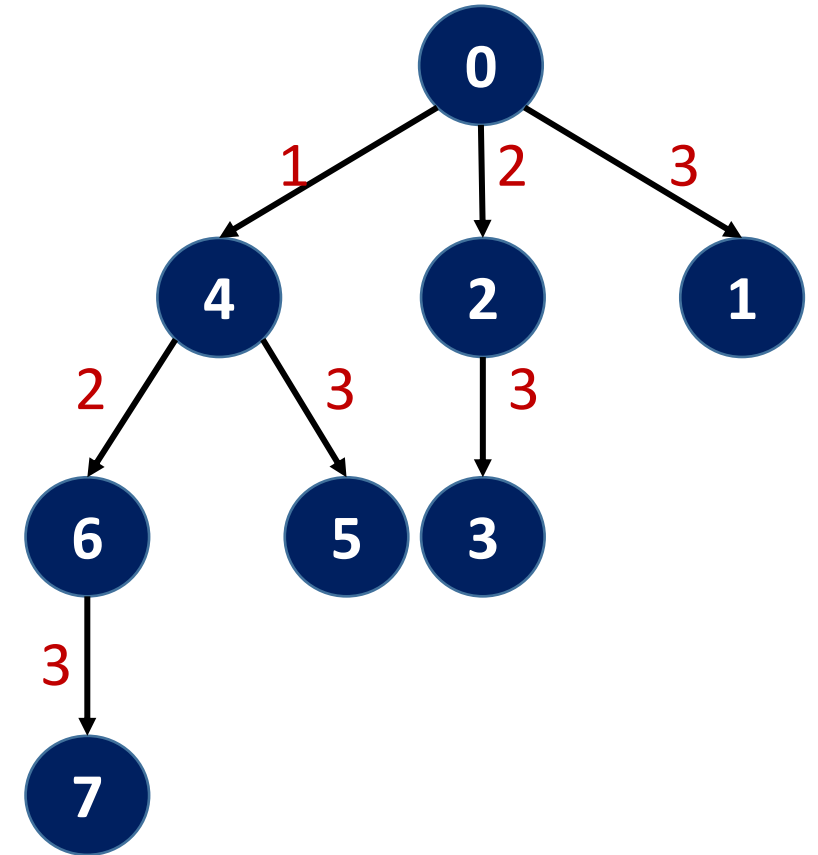
$2L + n/B$

0 -> 1, 2 -> 3, 4 -> 5, 6 -> 7

$L + n/B$

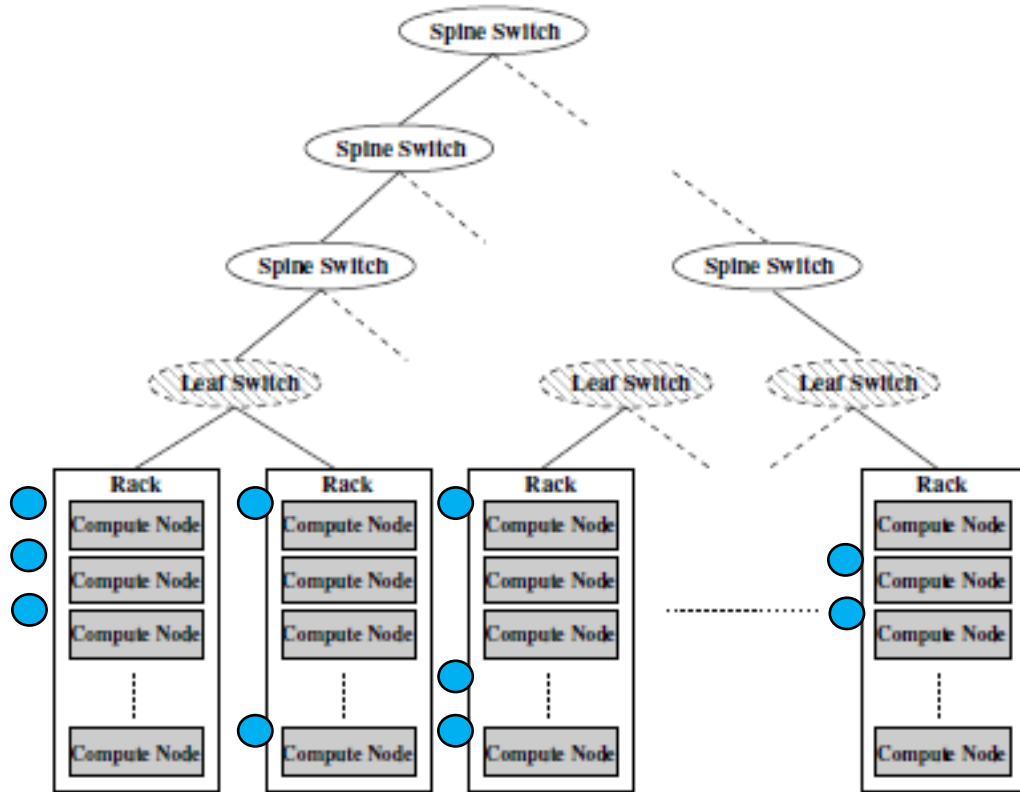
What all did we assume here?

$4L + (\log_2 8) n/B$



Cost of Communication

$$t_s\text{-intra-node} < t_s\text{-intra-switch} < t_s\text{-inter-switch}$$
$$t_w\text{-intra-node} < t_w\text{-intra-switch} < t_w\text{-inter-switch}$$



Cost involved for communication within the same node

L: $t_s\text{-intra-node}$

B: $t_w\text{-intra-node}$

Cost of communication within the same leaf switch

L: $t_s\text{-intra-switch}$

B: $t_w\text{-intra-switch}$

Cost involved for an inter-switch communication

L: $t_s\text{-inter-switch}$

B: $t_w\text{-inter-switch}$

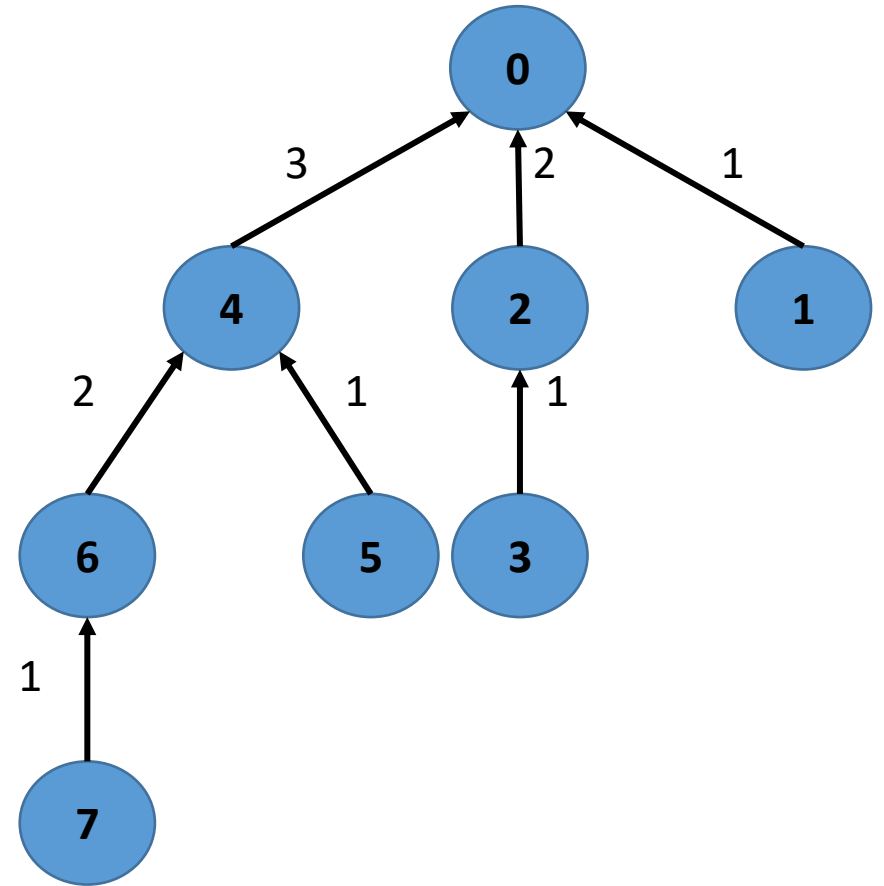
1. Actual cost depends on the #hops based on the actual placement of processes
2. Contention for intra-node/switch $\ll$ inter-switch

MPI_Gather Communication Cost

Binomial tree

Time: $(\log p) * L + (p-1)/p * (n/B)$

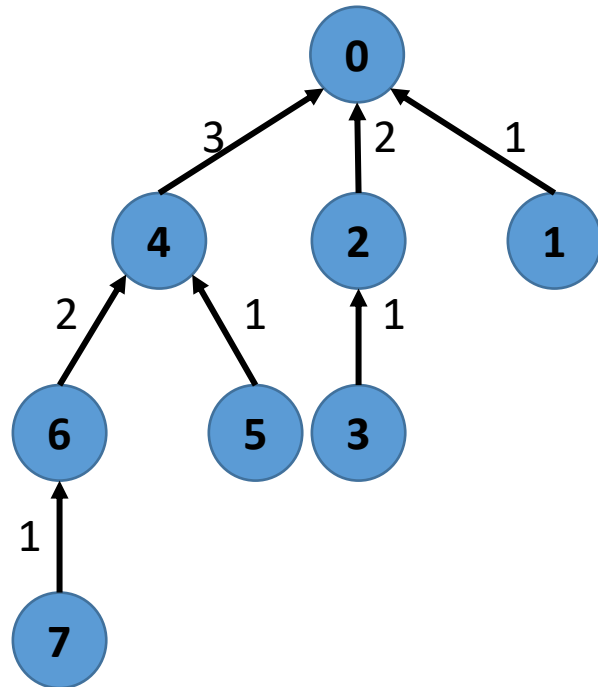
L = latency, B = bandwidth



Cost Model (Original MPI_Gather - Binomial)

Number of racks = R
Number of processes = P
Message size = N

Number of exchanges at i^{th} level: C_i
 C_1 = Number of intra-node transfers
 C_2 = Number of intra-switch transfers
 C_3 = Number of inter-switch transfers
 $C_1 + C_2 + C_3 = \log P$
Switch-level contention: α



There's a typo in the paper. Read this as ts-intra-node

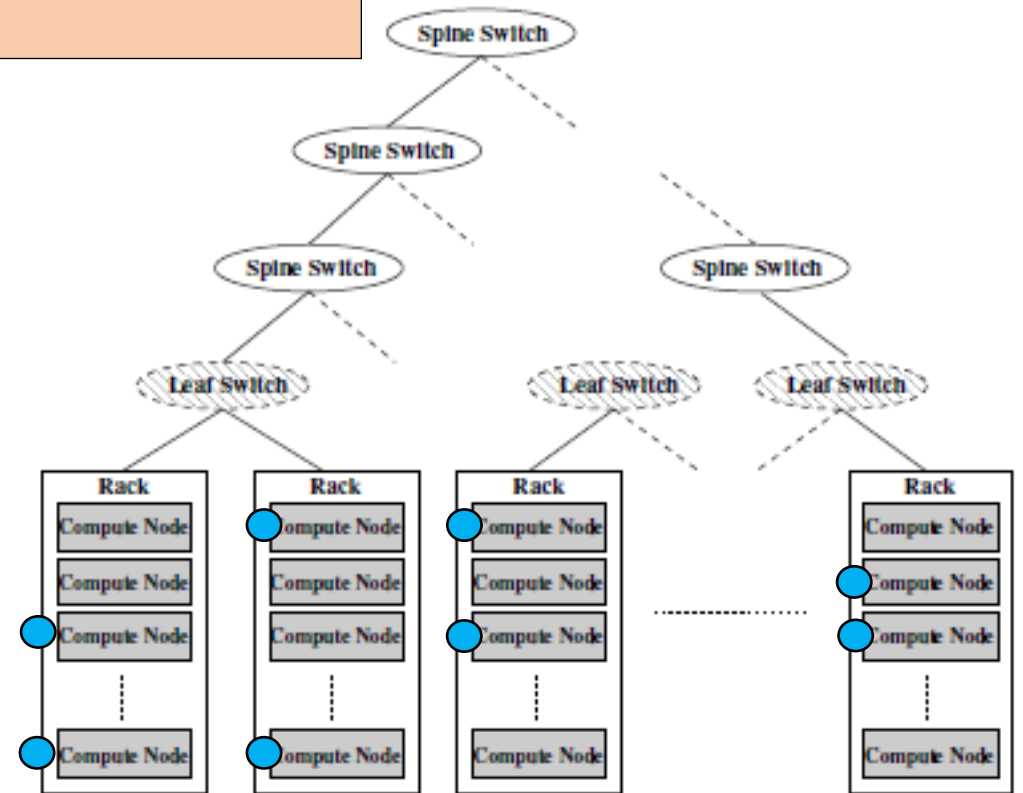
$$T_{binomial} = (t_{s-inter-node} * C_1 + t_{s-intra-switch} * C_2 + \alpha * t_{s-inter-switch} * C_3) + t_{w-intra-node} * (C_1) * (N * \gamma) + t_{w-intra-switch} * (C_2) * (N * \beta) + \alpha * t_{w-inter-switch} * (C_3) * (N * \delta)$$

Communication Cost for MPI_Gather (Binomial)

- The bandwidth term is obtained by adding costs at each level
- $[C_1 * \gamma + C_2 * \beta + C_3 * \delta] * N = (P - 1)/P * N$

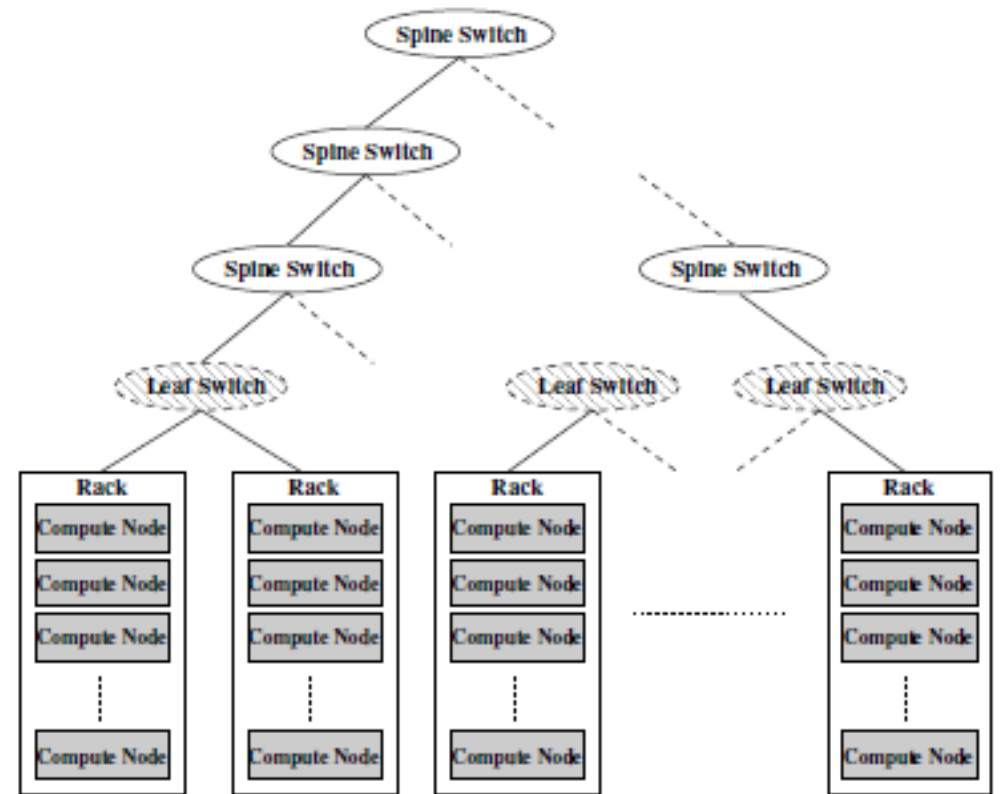
There's a typo in the paper. Read this as $t_{s-intra-node}$

$$T_{binomial} = (t_{s-inter-node} * C_1 + t_{s-intra-switch} * C_2 + \alpha * t_{s-inter-switch} * C_3) + t_{w-intra-node} * (C_1) * (N * \gamma) + t_{w-intra-switch} * (C_2) * (N * \beta) + \alpha * t_{w-inter-switch} * (C_3) * (N * \delta)$$



Topology-aware MPI_Gather

- Designate a rack leader process
- Rack-leader processes independently perform intra-switch gather
- R rack leaders perform inter-switch gather
- Reduced L and B terms (due to reduction in inter-switch exchanges)



Communication Cost Comparison

$$T_{binomial} > (\alpha * t_{s-inter-switch} * C_3) + (\alpha * N * t_{w-inter-switch} * C_3 * \delta)$$

$$T_{topo} > (\alpha * t_{s-inter-switch} * \log(R)) + (\alpha * (1 - 1/R)) / (M * t_{w-inter-switch})$$

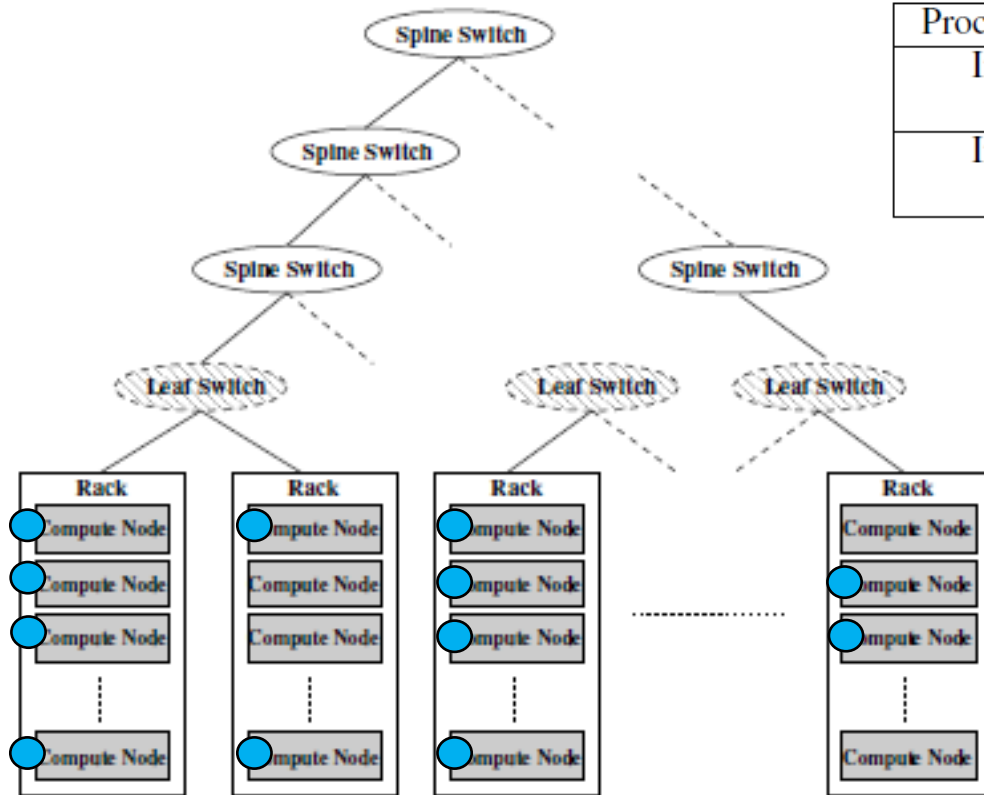
There's another typo in the paper !! Read / as*

$$T_{binomial}/T_{topo} = N * \delta * C_3 / (M * (1 - 1/R))$$

$$T_{binomial}/T_{topo} = f(R)$$

1. The rack-leader processes independently perform an intra-switch gather operation. This phase of the algorithm does not involve any inter-switch exchanges.
2. Once the rack-leaders have completed the first phase, the data is gathered at the root through an inter-switch gather operation performed over the R rack-leader processes.

Create Sub-communicators



Process Location		Number of Hops	MPI Latency (us)
Intra-Rack	Intra-Chassis	0 Hops in Leaf Switch	1.57
	Inter-Chassis	1 Hop in Leaf Switch	2.04
Inter-Rack		3 Hops Across Spine Switch	2.45
		5 Hops Across Spine Switch	2.85

- intra-chassis communicators (if used)
- intra-switch communicators
- chassis-leader communicators (if used)
- switch-leader communicators

Discover Topology

- Infiniband tools
 - ibnetdiscover – outputs the switch connections / device identifiers
 - One-time discovery (in general)
- MPI_Init
 - Create intra-chassis communicators – all nodes in the same chassis
 - Create intra-switch communicators – all nodes in the same leaf switch
 - Assign one chassis-leader and one switch-leader
 - Create switch-leader and chassis-leader communicators

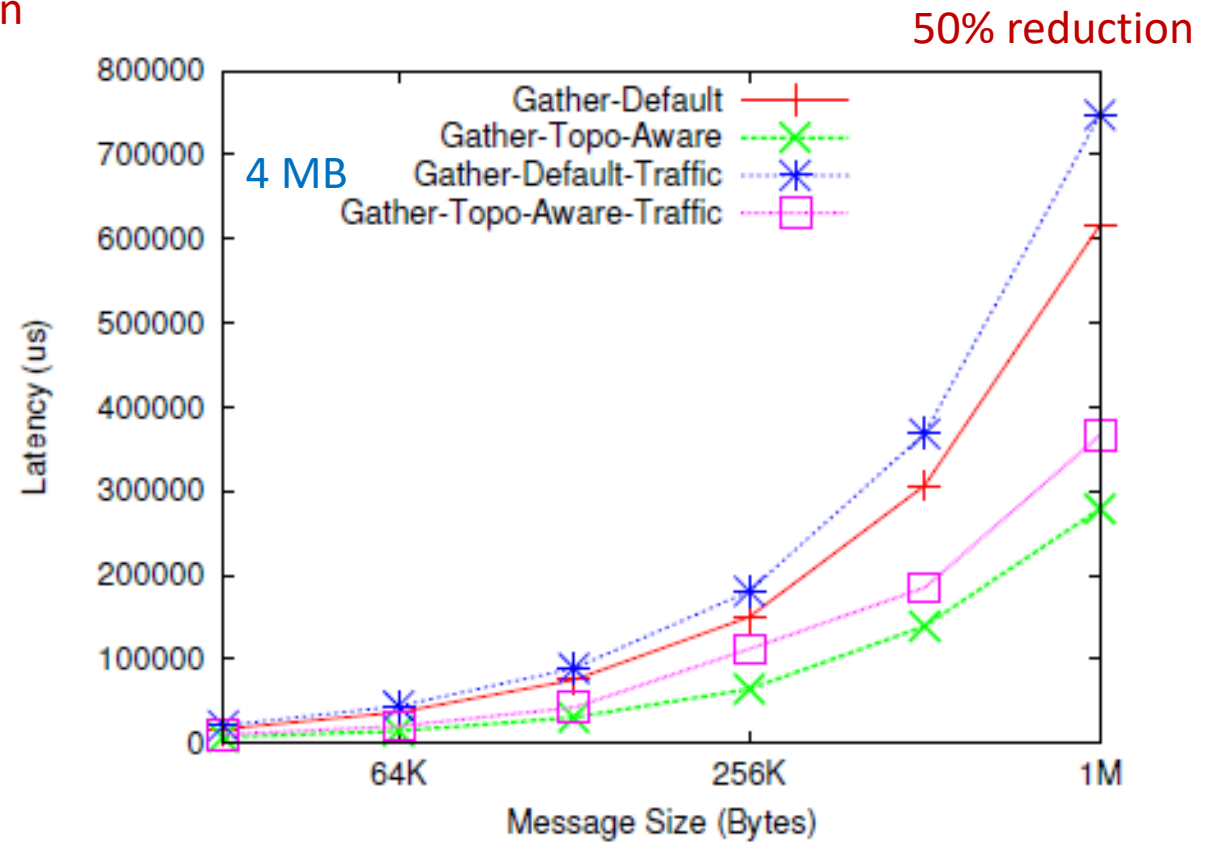
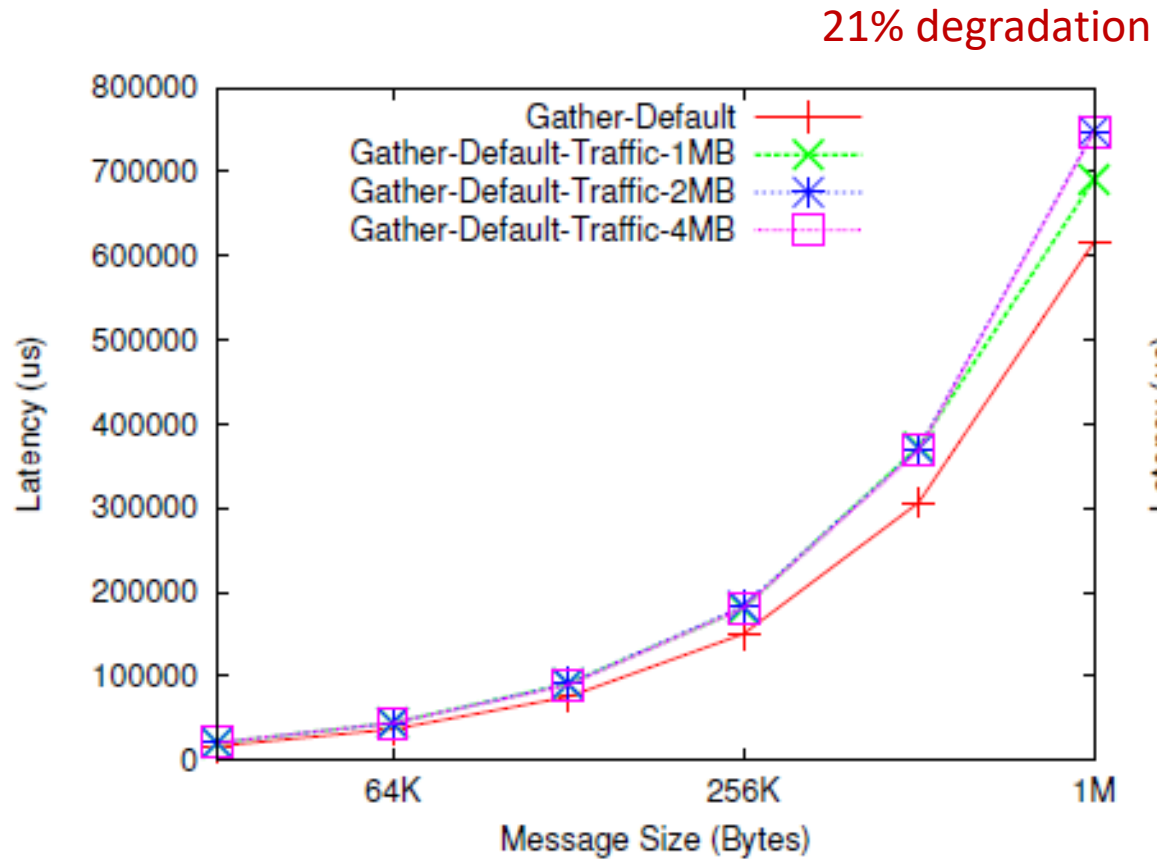
Experimental Setup

- Three InfiniBand DDR switches A, B and C to create a tree topology.
- Switch A is connected to 8 nodes based on the quad-core, quad-socket AMD Barcelona architecture (4 nodes /64 processes used)
- Switch B is connected to 32 nodes based on the quad-core, dual-socket Intel Clovertown architecture (29 nodes/ 232 processes used)
- Switches A and B are connected to Switch C with two InfiniBand DDR links each.

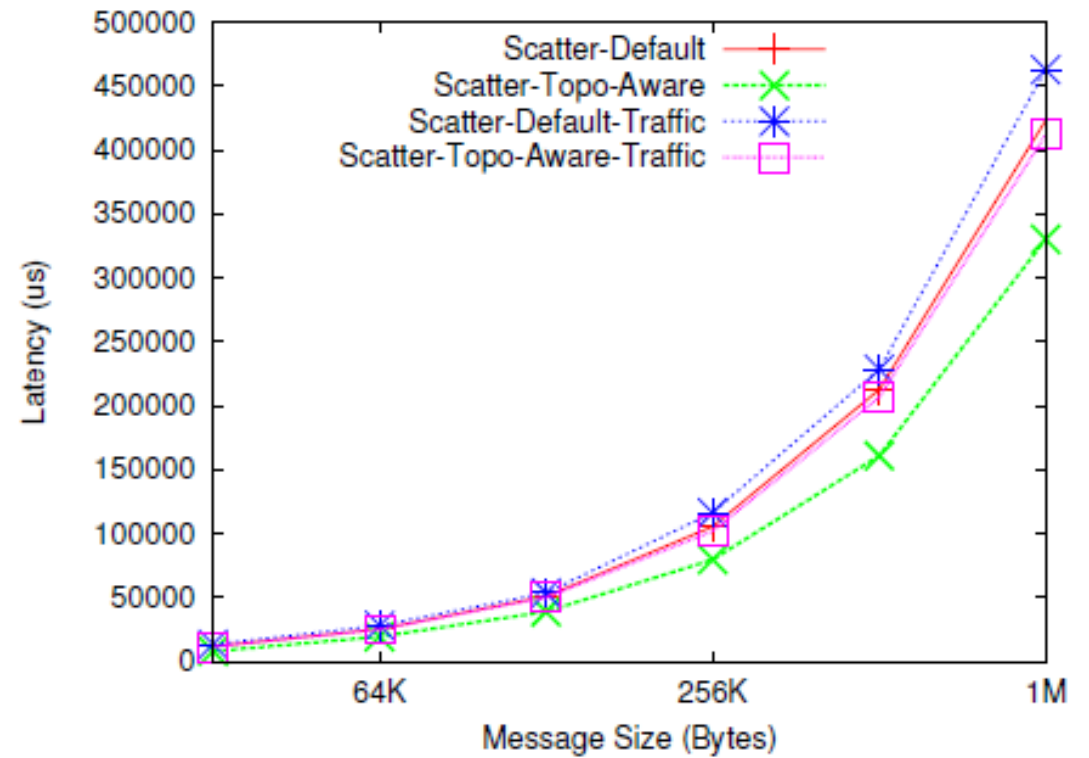
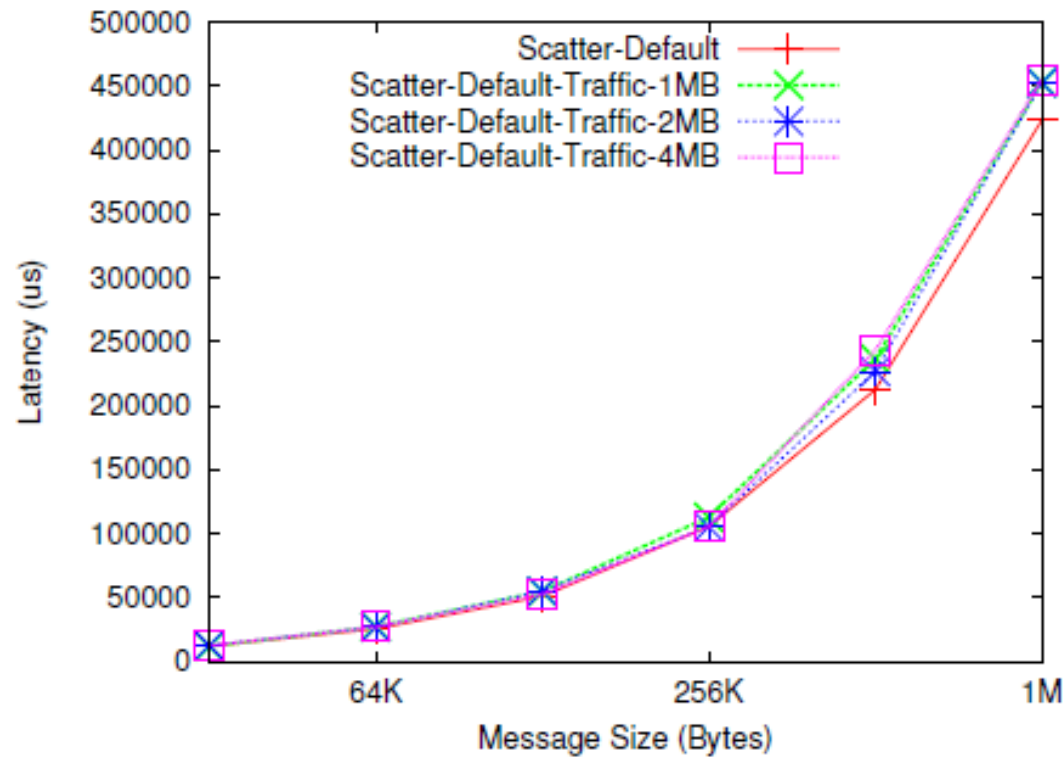
Benchmark

- A simple benchmark code iterates through various message sizes (0 – 1 MB) and invokes a collective call several times in a loop.
- Measured average time when system is quiet
- MPI_AlltoAll is used to create background traffic ($K/2$ in each switch)
 - Synthetic benchmark
 - Constant traffic $K * M$ bytes over the switches
 - This creates interference for the MPI_Gather/MPI_Scatter job

Gather Results (With and Without Traffic)



Scatter Results

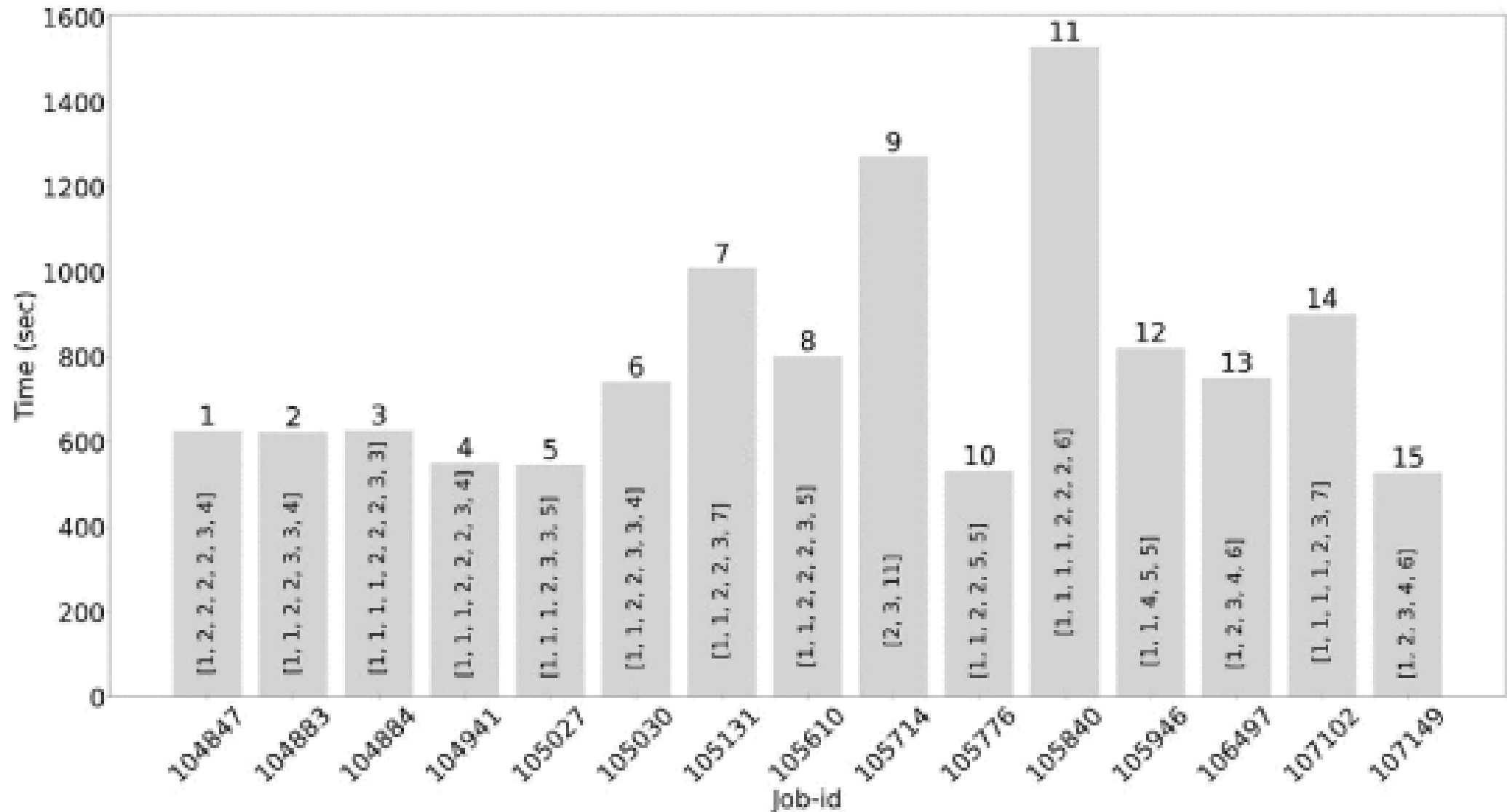


Conclusions

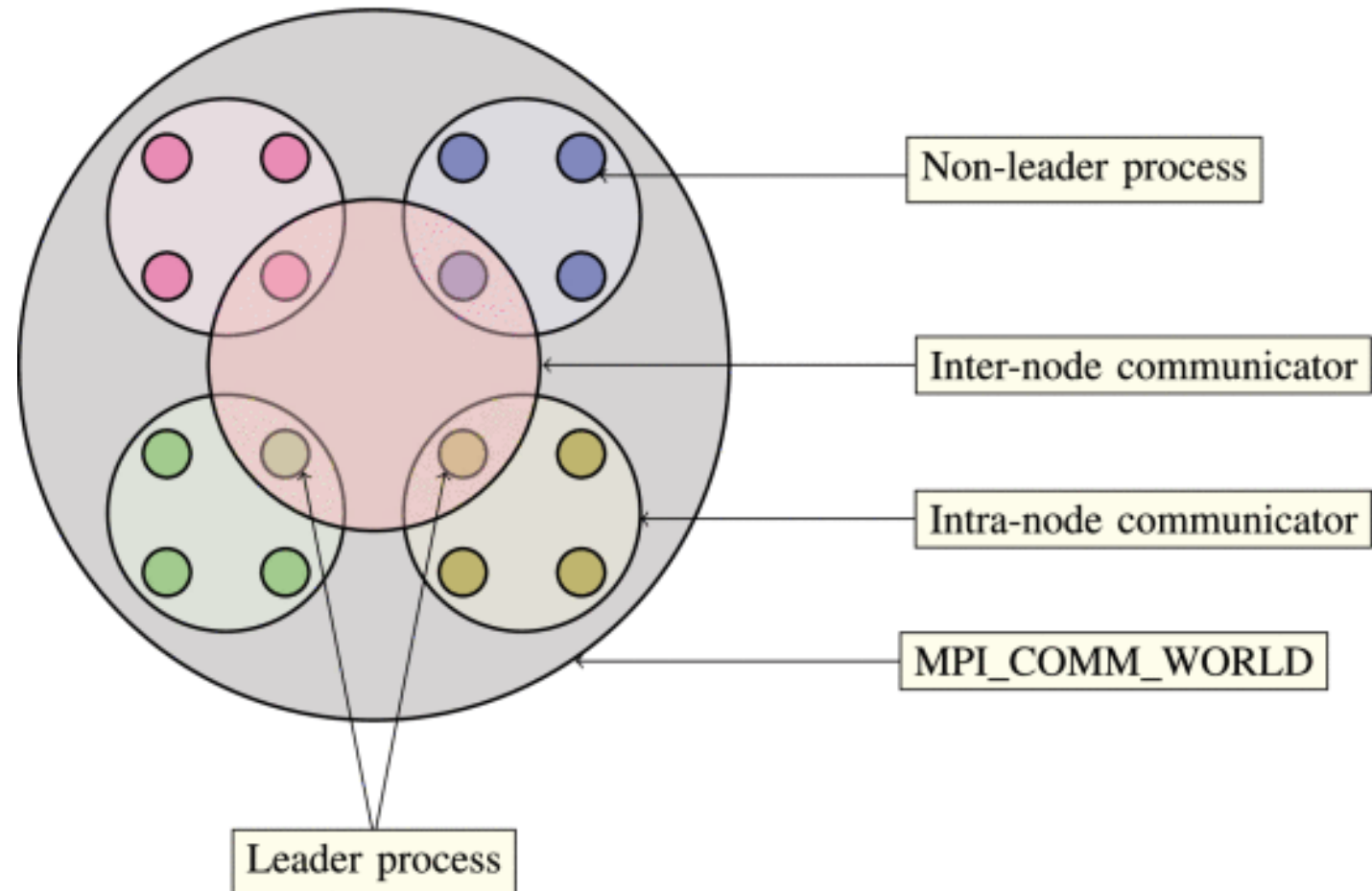
- Presence of background traffic may lead to performance degradation
- Efficient software stack may improve performance
- Topology-aware MPI_Gather and MPI_Scatter was shown to perform better

Hierarchical Communication Optimization for
FFT, M Kumar, P Malakar, SC W HiPar 2022

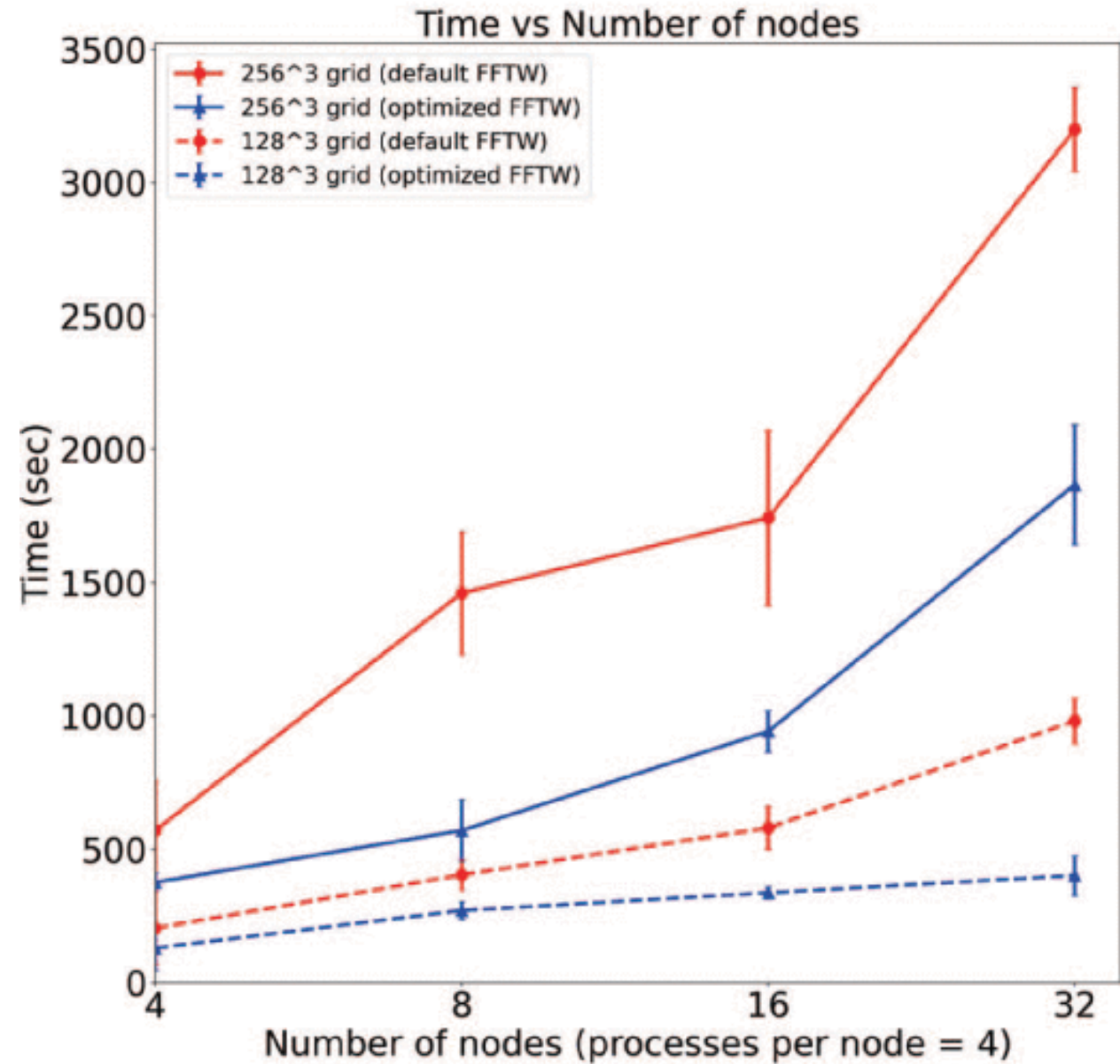
Variation in Job Runtimes



Hierarchical AlltoAll/Sendrecv



Results

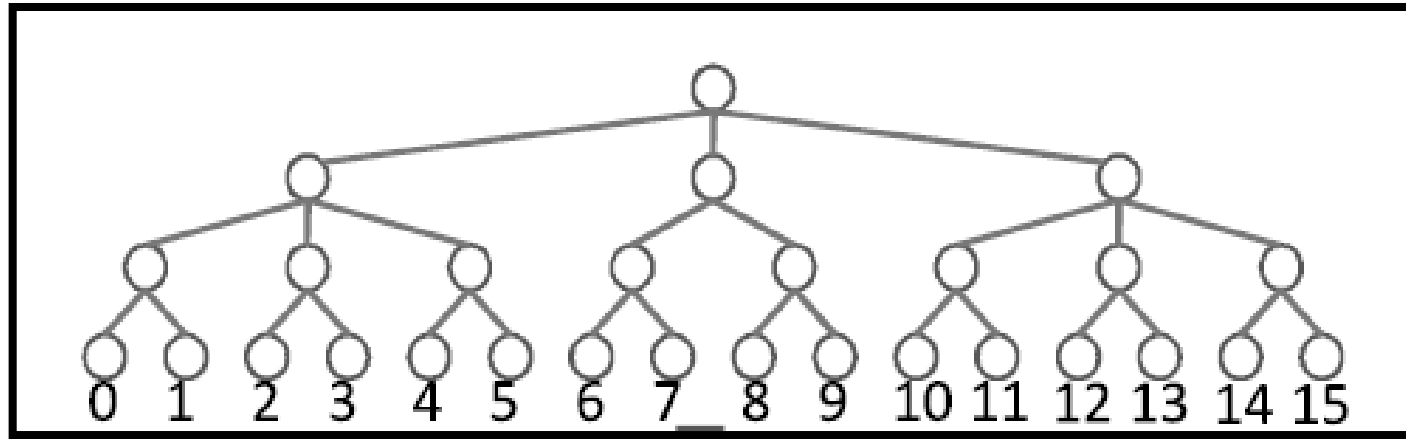


Network Performance Aware MPI Collective
Communication Operations in the Cloud
Gong et al., IEEE TPDS 2015

Algorithm

- Network performance among a set of VMs
 - Latency matrix
 - Bandwidth matrix
 - Network performance matrix (Hockney model)
- Group the machines
 - Form a number of non-overlapping group
 - Each group has machines that have similar network performance (use a closeness parameter)
 - Build hierarchy among the groups by increasing the closeness parameter

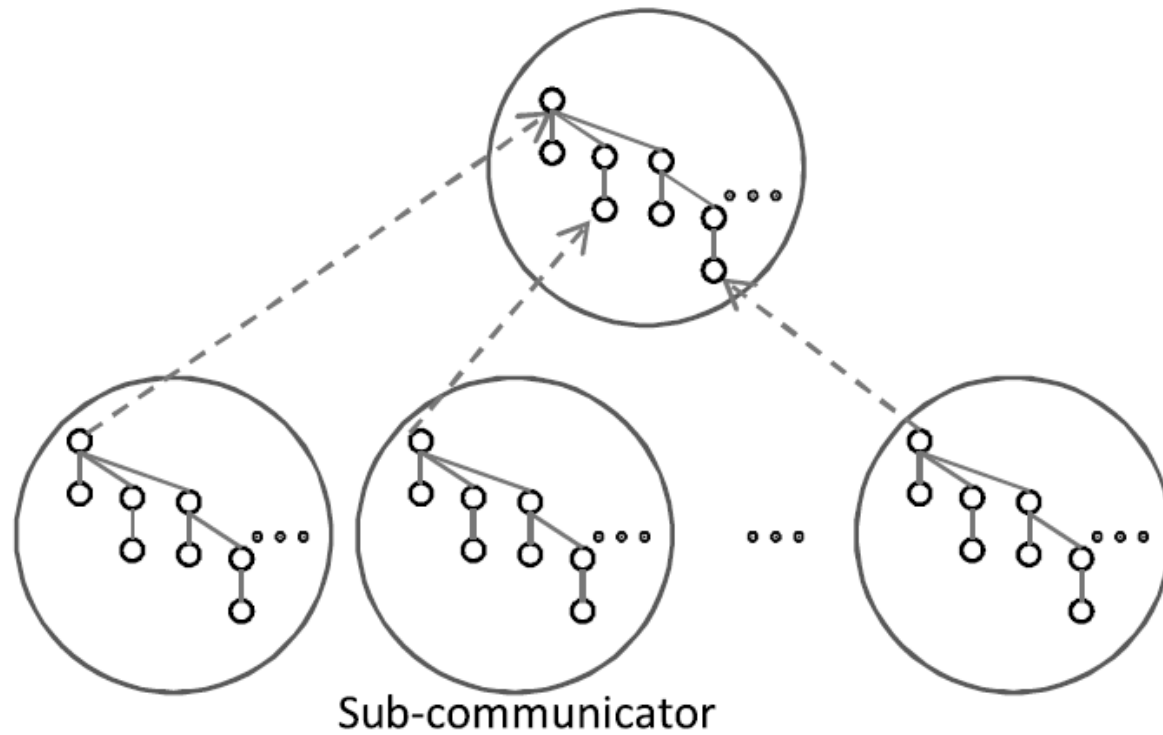
Network-aware Hierarchy



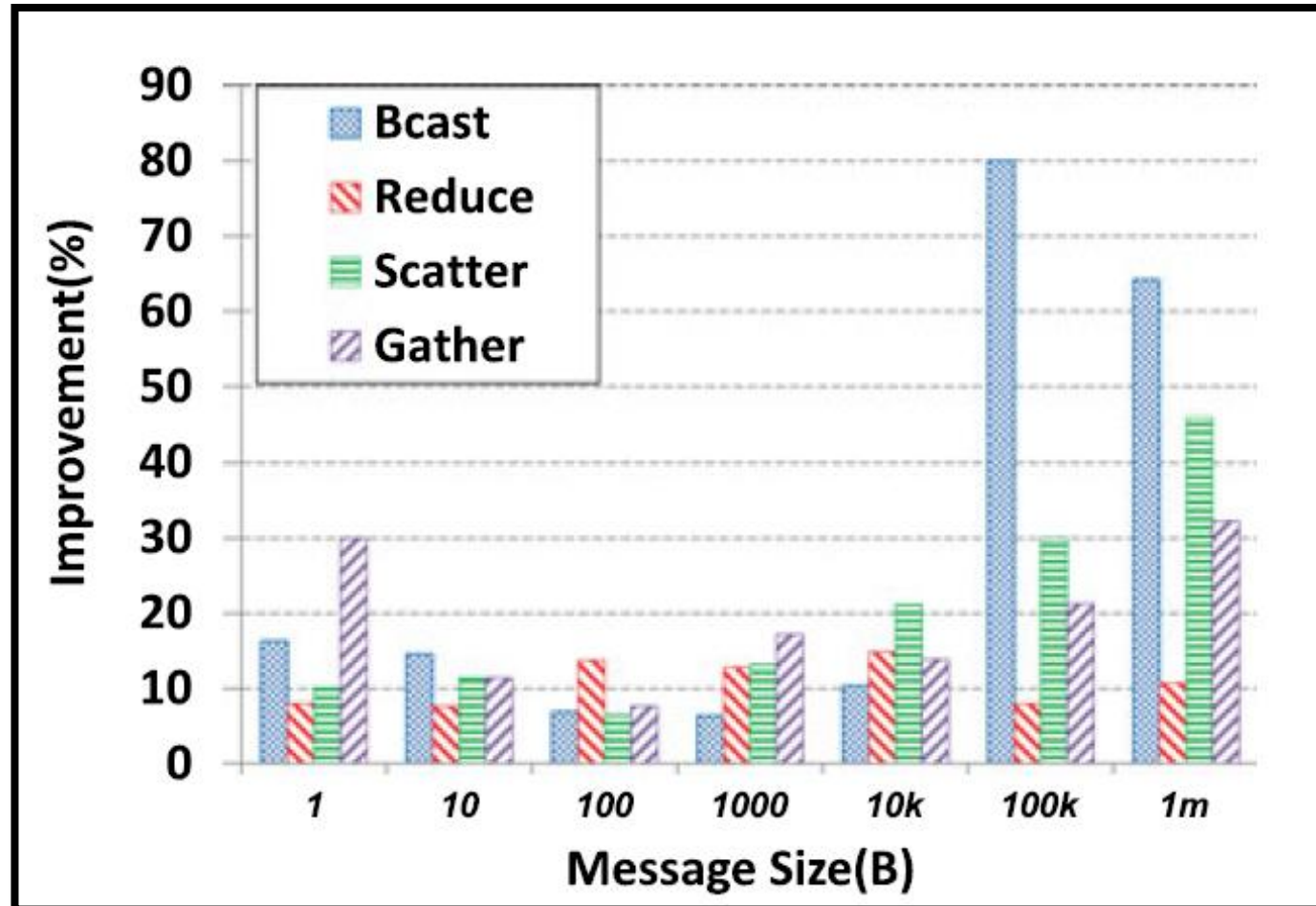
Network Performance Aware Reduce

- Build network performance hierarchy based on L , B , m (message size)
- Select leader process in groups in G_0
- Create sub-communicators for each group
- Each group performs reduce independently
- For each level l
 - Select leader among leaders in G_l
 - Create sub-communicator for each group
 - Perform reduce independently

Network-aware Reduce



Results

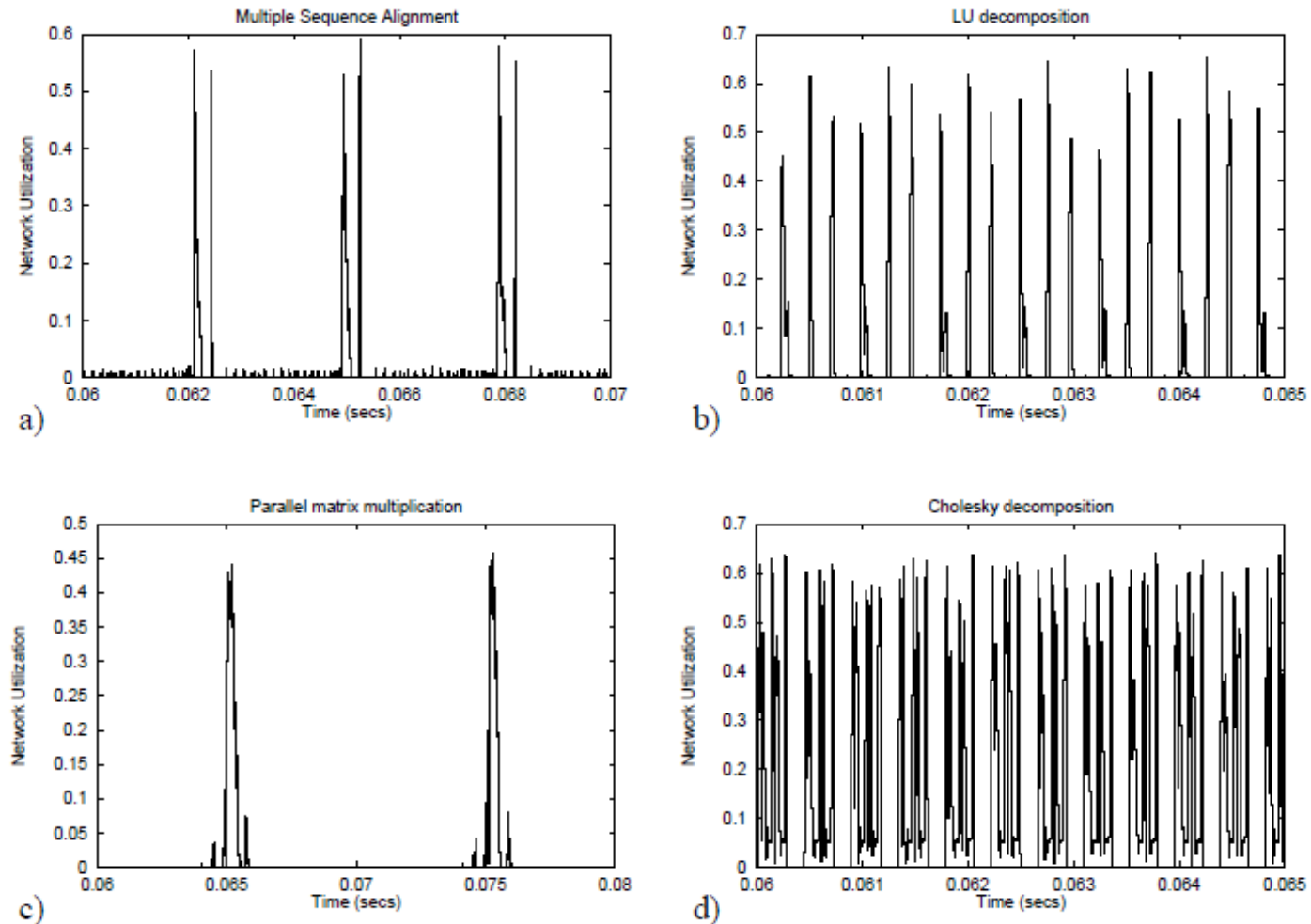


Resources Required for Node Allocation (Recap)

- Number of nodes
- Wall-clock time
- Users are charged for node-hours/core-hours

What is missing?

Network Utilization in Different Applications



Network Utilization in FFT

